

git

Prácticas — parte 3: repositorios remotos

Antes de empezar

- A diferencia de las partes 1 y 2, **esta vez sí necesitas** una cuenta de GitHub y conexión a internet.
- Vas a retomar el proyecto `practica-ramas` de la parte 2. Si ya no lo tienes, el ejercicio 1 te indica cómo recrearlo en 2 minutos.
- Tendrás que autenticarte contra GitHub con un **token (PAT)** o con **SSH** — no con tu contraseña, ya no funciona.
- Ve resolviendo los ejercicios en orden, cada uno se apoya en el anterior.
- Los **retos** (al final) no traen los pasos, solo el objetivo: decides tú los comandos.

Ejercicios guiados

Ejercicio 1 — Recupera (o recrea) tu proyecto

Si todavía tienes la carpeta `practica-ramas` de la parte 2, entra en ella y pasa al ejercicio 2.

Si no la tienes, créala rápido:

```
mkdir practica-ramas && cd practica-ramas
git init
printf '<h1>Mi Portfolio</h1>\n<p>Sobre mi</p>\n' > index.html
printf 'body { font-family: sans-serif; }\n' > styles.css
git add .
git commit -m "Commit inicial del portfolio"
```

Resultado esperado: `git log --oneline` muestra al menos 1 commit en `main`.

Ejercicio 2 — Crea tu cuenta y autentícate

1. Si no tienes cuenta, créala en <https://github.com/signup>
2. Configura una forma de autenticarte por línea de comandos:
 - **Token (PAT):** *Settings* → *Developer settings* → *Personal access tokens*, genera uno con permiso de `repo`.
 - **SSH:** genera un par de claves y añade la pública a tu cuenta de GitHub.
3. Compruébalo clonando un repositorio público cualquiera en una carpeta temporal.

Resultado esperado: el `git clone` de prueba termina sin errores de autenticación.

Ejercicio 3 — Crea el repositorio remoto

1. En GitHub, crea un repositorio nuevo llamado `practica-ramas`.
2. **No lo inicialices** con `README.md` ni `.gitignore` (ya tienes commits en local; si el remoto también tiene commits propios, las dos historias no encajarán a la primera).
3. Copia la URL que te da GitHub (HTTPS o SSH, según lo que configuraste en el ejercicio 2).

Resultado esperado: tienes un repositorio vacío en GitHub, listo para recibir tu historial local.

Ejercicio 4 — Conecta tu repo local con GitHub

Desde `practica-ramas`:

```
git remote add origin <url_que_copiaste>  
git push -u origin main
```

Resultado esperado: al recargar la página del repositorio en GitHub, ves tu `index.html`, `styles.css` y el historial de commits.

Ejercicio 5 — `git remote`

1. Comprueba tu remoto configurado: `git remote -v`.
2. Añade un segundo remoto de prueba apuntando a cualquier URL (no hace falta que sea válida): `git remote add prueba https://github.com/octocat/Hello-World.git`
3. Vuelve a listar con `git remote -v` y comprueba que aparecen los dos.
4. Elimina el remoto de prueba: `git remote remove prueba`.

Resultado esperado: `git remote -v` vuelve a mostrar solo `origin`.

Ejercicio 6 — Clona tu propio repositorio (simula "otro ordenador")

1. Sal de la carpeta actual y clona tu repositorio en una carpeta distinta:

```
cd ..  
git clone <url_de_tu_repo> practica-ramas-clon  
cd practica-ramas-clon
```

2. Comprueba que el remoto ya está configurado automáticamente: `git remote -v`.
3. Comprueba las ramas remotas disponibles: `git branch -r`.

Resultado esperado: `origin` ya apunta a tu repositorio, sin que hayas ejecutado `git remote add` esta vez.

Ejercicio 7 — Sube una rama nueva y tráela en la otra copia

1. En **practica-ramas** (la carpeta original), crea una rama, añade un footer a `index.html`, confírmalo y súbelo:

```
git checkout -b feature/pie-pagina
# añade <footer>Hecho con git</footer> antes de </body> en index.html
git add .
git commit -m "Añado pie de página"
git push -u origin feature/pie-pagina
```

2. En **practica-ramas-clon**, trae la información de las ramas remotas: `git fetch`.
3. Comprueba con `git branch -r` que aparece `origin/feature/pie-pagina`.
4. Pásate a esa rama: `git checkout feature/pie-pagina`.

Resultado esperado: en **practica-ramas-clon** ves el footer, sin haberlo escrito tú ahí.

Ejercicio 8 — `git fetch` vs `git pull`

1. Desde `practica-ramas-clon`, en `main`, añade una pequeña mejora (por ejemplo, un comentario HTML), confírmala y súbela.
2. Desde `practica-ramas` (la original), ejecuta `git fetch origin` y comprueba con `git log --oneline main..origin/main` que el cambio ya está disponible en el remoto... pero tu `index.html` local **todavía no lo tiene**.
3. Ahora ejecuta `git pull origin main` y comprueba que tu `index.html` local se ha actualizado.

Resultado esperado: entiendes que `fetch` descarga sin tocar tus ficheros, y `pull` descarga y fusiona.

Ejercicio 9 — Provoca (y resuelve) un push rechazado

1. Desde **ambas copias**, sitúate en `main` y haz un cambio distinto en cada una (por ejemplo, edita una línea distinta de `styles.css` en cada carpeta) y confírmalo en las dos, **sin sincronizar entre medias**.
2. Haz `git push` desde una de las dos copias: debería funcionar.
3. Haz `git push` desde la otra: debería fallar con `[rejected]`.
4. Resuélvelo: `git pull origin main` (resuelve el conflicto si aparece) y vuelve a hacer `git push`.

Resultado esperado: sabes reaccionar ante un push rechazado sin recurrir a `--force`.

Ejercicio 10 — Tu primer Pull Request

1. Desde `practica-ramas`, crea una rama `feature/seccion-contacto`, añade una sección de contacto a `index.html`, confírmalo y súbelo con `-u`.
2. Ve a GitHub y abre un **Pull Request** de `feature/seccion-contacto` hacia `main`, con un título y una descripción claros.
3. Fusiona el Pull Request desde la interfaz web de GitHub.
4. Vuelve a la terminal, sitúate en `main` y haz `git pull`.

Resultado esperado: el cambio que fusionaste en GitHub aparece ahora en tu `main` local, sin que hayas hecho `git merge` tú mismo en la terminal.

Retos

Aquí no hay pasos: piensa qué comandos necesitas.

Reto 1 — Conflicto en remoto

Objetivo: repite la situación del ejercicio 9, pero esta vez haz que ambas copias modifiquen **la misma línea** del mismo fichero. Resuelve el conflicto que aparezca al hacer `git pull`, y deja ambas copias sincronizadas.

Reto 2 — Fork y `upstream` de verdad

Objetivo: busca un repositorio público pequeño en GitHub (puede ser el de un compañero), haz un fork, clónalo, y configura `upstream` apuntando al original. Simula que el original avanza (pide a su dueño que haga un commit, o usa uno que ya exista) y trae esos cambios a tu fork con `fetch / merge`.

Reto 3 — Pull Request cruzado

Objetivo: con un compañero, dale acceso de colaborador a tu repositorio `practica-ramas` (o usa el suyo). Crea una rama, haz un cambio, abre un Pull Request real, y pide a tu compañero que lo revise y lo fusione desde GitHub.

Reto 4 — Elige el modelo de ramas

Objetivo: para cada uno de estos proyectos, decide si encaja mejor con GitFlow, GitLab Flow o Trunk-Based Development, y justifica por qué:

- Una app web que despliega a producción varias veces al día.
- Una aplicación de escritorio que publica una versión nueva cada 6 meses y da soporte a las 2 versiones anteriores.
- Un equipo pequeño que despliega directamente sobre `main` usando *feature flags*.

Reto 5 — Rota tu token de acceso

Objetivo: revoca el token (PAT) que generaste en el ejercicio 2 desde GitHub, genera uno nuevo, y comprueba que sigues pudiendo hacer `git push` con el nuevo token (tendrás que volver a autenticarte).

Autoevaluación

¿Te ves capaz de...?

- [] Clonar un repositorio remoto y conectar uno local a GitHub sin usar `git clone`.
- [] Explicar la diferencia entre `git fetch` y `git pull`.
- [] Resolver un `push` rechazado sin recurrir a `--force`.
- [] Explicar qué es un fork y para qué sirve el remoto `upstream`.
- [] Crear y fusionar un Pull Request desde la interfaz de GitHub.
- [] Leer el nombre de una rama remota (`origin/rama`) y saber qué significa.
- [] Explicar la diferencia entre autenticarte con un token (PAT) y con SSH.
- [] Elegir un modelo de ramas adecuado según el tipo de proyecto.